Proof of Repair: VIN-Bound Tamper-Evident Repair Records from LLM-Assisted Vehicle Diagnosis

Shivansh Sharma
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
shivansh.sharma2023@vitstudent.ac.in

Himali Sharma
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
himali.sharma2023@vitstudent.ac.in

Vikash Dhillan
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
vikash.dhillan2023@vitstudent.ac.in

Thanuja R
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
thanuja.r@vit.ac.in

***Abstract*—The resale value and safety of a used vehicle are heavily influenced by its repair history, which is typically documented only through paper invoices, workshop stamps, and unsigned PDF service reports. These documents can be altered, back-dated, or selectively withheld, leaving buyers, dealers, or insurers with little reliable proof that a declared repair actually occurred, that it was performed at the stated odometer reading, or that the supporting paperwork remains unchanged. This paper presents a cloud-based Proof-of-Repair system that creates a verifiable, tamper-evident repair history linked to a vehicle identification number (VIN). The workshop-captured data, including the VIN, vehicle details, OBD-II diagnostic trouble codes, symptoms, sensor notes, and odometer reading, is processed by a large language model fine-tuned with parameter-efficient LoRA for automotive fault diagnosis, which outputs a structured diagnosis and a ranked repair procedure. Supporting evidence, including invoices, photographs, and reports, is stored off-chain, while only digests and content identifiers are retained. Each repair is recorded as a Proof-of-Repair leaf associated with the vehicle identification number, vehicle details, odometer reading, report hash, evidence identifier, and model provenance. Leaves are organized into a Merkle tree, with its root secured by an EVM-compatible smart contract and independently timestamped via OpenTimestamps, ensuring that any alteration to a committed record invalidates its digest, updates the recomputed root, and triggers detection. A buyer, dealer, or insurer checks the repair timeline for a vehicle identification number and verifies the sealing status of every event. Two fine-tuned models were evaluated on a real-world OBD hold-out set containing 57 cases, achieving fault-code accuracy rates of 93. 0% and 98. 3%. This system is a research prototype that operates on a local EVM development chain, uses synthetic data for demonstration, and defers physical OBD-II integration, production deployment, and large-scale field validation to future work.**

Keywords*—*proof of repair, second-hand vehicles, vehicle repair history, VIN, OBD-II diagnostic trouble codes, large language models, LoRA fine-tuning, Merkle tree, blockchain anchoring, OpenTimestamps, data integrity, and content-addressed evidence.

1. Introduction

Used cars are sold on the basis of documents that cannot be checked. A buyer gets a folder of invoices, a stamped service book and a printed job card, and is asked to accept that the work described was done, on this vehicle, at the mileage shown. None of these documents have any evidence of when it was created or if it has been altered since, and the records that would contradict them can simply be withheld. Akerlof described this asymmetry as the condition for markets in goods of uncertain quality [1], and it is still not solved in practice [2].

The price is reflected in odometer tampering. In December 2025, 2.45 million cars on U.S. roads had odometers which had been rolled back, a 14 percent increase in one year, and buyers of these cars lost an average of $3,300 each, according to CARFAX estimates [47]. The national regulator estimates that around 450,000 vehicles are sold each year with faked readings [48]. Digital instrument clusters can be rewritten with cheap tools, so the falsification leaves no trace on the instrument that it falsifies [47]. Detection therefore has to come from records outside the vehicle, and those records are precisely what cannot be trusted at present.

There are two capabilities, and the common mistake is that of confusing the two. The first is interpretation: taking diagnostic trouble codes, symptoms and sensor observations and turning them into a structured statement of what is wrong and what needs to be done [29], [30]. The second is integrity. Once a repair has been sealed, the garage that made it and anyone who works on it afterward cannot change the record or the evidence that supports it without notice. The first is a language problem, solved here with a model adapted by low-rank adaptation [10]. The second is a cryptographic problem, solved using content-addressed evidence, Merkle commitments [20] and public anchoring [23], [35].

This paper describes a prototype that joins them, and contributes four things. A canonical Proof-of-Repair leaf hashes together a vehicle identification number, odometer reading, report digest, evidence identifier and model provenance. A batching path pins those leaves to an EVM contract and separately to a public time-stamping service. A diagnostic layer for automotive fault codes is evaluated over real OBD cases, including the failure conditions. Finally, a demonstration run against the implementation shows that an odometer rollback, an invoice substitution and the reuse of a record across vehicles are each detected.

The role segregation continues to hold. The model returns records that are not guaranteed to be accurate. The cryptographic layer does not judge whether the records were right, but makes it impossible to modify the records after they have been sealed. What the system provides is a repair history a stranger can verify, not a diagnosis a stranger should trust.

2. Related Work

Four bodies of work meet in this problem, and none of them spans it.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

The economic case is settled. Akerlof showed that when buyers cannot distinguish the quality, price tends to the average and honest sellers exit [1]. This argument has been extended to vehicle markets and trust-free transaction systems [2]. But what the literature lacks is a mechanism for generating evidence that a buyer can actually verify.

The input vocabulary is vehicle diagnosis. Diagnostic services and trouble codes are standardized [29], [30], as are vehicle identification numbers [28]. This gives any system a stable key and a shared fault language. This is extended to prognosis by data-driven work on run-to-failure datasets and remaining-life estimation [18], [19], with wider surveys of machinery health and automotive predictive maintenance [7], [8]. That work stops at the inference; what happens to the inference after that is out of its scope.

The interpretation is provided by language models. Transformer architectures transform unstructured technical description to structured output [13], [14], [15], and instruction tuning improves adherence to a requested form [16]. The known weakness is reliability. Fluent output that is incorrect is a documented failure mode [9], so any system that commits model output must treat the commitment and the correctness as separate claims. Adapter layers [11], low-rank adaptation [10] and quantized adaptation [12] train a small set of parameters against a frozen backbone, making adaptation affordable. Open-weight models of modest size [26], [27] put this within reach of ordinary hardware.

Integrity adds the missing ingredient. Public ledgers provide an append-only state with no single controller [23], programmable chains extend this to arbitrary state transitions defined in code [24], and vehicular proposals apply to telemetry, forensics and ownership [3], [4], [6], [22]. Merkle trees offer a summary of many records in a single hash while preserving membership proofs [20], [31], the construction underlying certificate transparency [32], [33] and supply-chain transparency work [34]. Timestamping makes it possible to prove the existence of a document at a certain point in time without trusting the timestamping party [5], [35], and content addressing allows the evidence to stay off-chain but still tamper-evident [25].

The gap is the seam, not any individual part. Diagnostic systems generate text and leave its custody unaddressed. Vehicular ledgers anchor telemetry or ownership, not repair events and their supporting documents. Transparency logs assume a committed object with structure and meaning agreed. No reviewed system commits a diagnosis, the repair performed in response to the diagnosis, the evidence of the repair, and the vehicle it belongs to in one commitment a stranger can verify. Thus, the first design problem is to define that committed object, and it comes before the anchoring mechanism in what follows.

3. System Design

The architecture is organized as a pipeline of layers, as illustrated in Figure 1. Raw diagnostic data from the vehicle enters at the top and exits at the bottom as a verifiable timeline presented to the buyer. The color coding distinguishes the four types of components: ordinary off-chain cloud services, the AI diagnostic layer, storage, and the on-chain and public-timestamp layer.

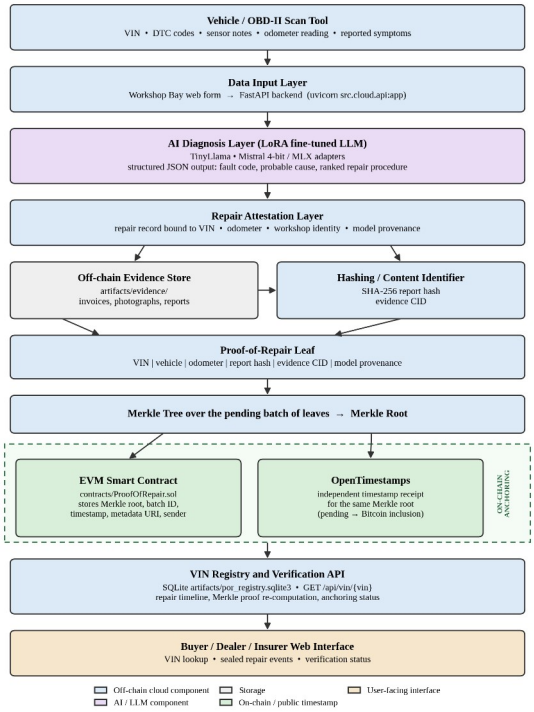


Figure 1. System architecture.

The data input layer receives the VIN, vehicle details, diagnostic trouble codes, reported symptoms, sensor notes, and odometer reading via the workshop bay form, which then sends this information to the FastAPI backend. The AI diagnosis layer forwards this context to a LoRA-adapted language model, which returns a structured diagnosis including the identified fault, its likely cause, and a prioritized repair procedure. The repair attestation layer integrates the diagnosis with the technician's actual work, linking it to the vehicle identification number, odometer reading, workshop identity, and the provenance of the model that generated the diagnosis.

Evidence follows a separate path. Invoices, photographs, and reports are stored off-chain, while only a summary of the repair report and a content identifier for each evidence item proceed to the commitment stage. The Proof-of-Repair leaf is subsequently formed. Leaves are accumulated into a pending batch; once the batch is anchored, a Merkle tree is constructed over it and its root is submitted to both the smart contract and OpenTimestamps. The VIN registry stores repair events, leaves, batches, and anchoring metadata, and delivers them via the verification API to the buyer-facing interface.

3.1 The Proof-of-Repair Leaf

The leaf is the unit of commitment, and the contents are fixed by the implementation, not by choice of presentation. Eleven fields are included in the hashed payload: a proof identifier, the vehicle identification number, the vehicle description, the odometer reading, the event type, the issuing workshop, the sealing timestamp, the digest of the repair report, the content identifier of the stored evidence, a model provenance record, and an open extension map. Everything that a later dispute might turn on is therefore inside a single digest, and nothing that would identify a person is.

Three of these merit a comment. The event type is chosen from a closed set of five values: diagnosis, repair, parts replacement, pre-purchase inspection and fault-code clearance. This prevents a diagnosis from being later presented as a completed repair. The vehicle identification number is converted to upper case before hashing, removing a trivial

source of disagreement between two parties that recorded the same vehicle differently. The extension map is always present in the payload, even if empty, to avoid a record being augmented after sealing in a way that would not be obvious.

Model provenance is a field without a counterpart in a paper service book. It records the base model, the adapter version, a digest of the adapter weights themselves, and a Merkle root over the training dataset manifest. A sealed diagnosis is thus attributable not just to a model by name, but to the exact parameters that produced it and the exact corpus those parameters were trained on. If a model version is found in the future that incorrectly labels a group of fault codes in a systematic manner, then all the records generated by it can be sought out and re-verified, a property the diagnostic layer cannot provide by itself.

Both the report and the evidence are shrunk before they enter the leaf. The repair report is serialized canonically and hashed; the evidence is written to a content-addressable store whose identifier is the digest of the stored bytes. The leaf thus commits to a diagnosis of several hundred words and to invoices and photographs of arbitrary size through two fixed-length values. A report of nine fields and an evidence payload of 1,966 bytes is represented by thirty-two bytes each in the record examined later in this paper.

Table 1. Fields bound in the hashed leaf payload

Field	Why it is bound
proof_id	Identifies the sealing event itself
vin	Binds the record to one vehicle; upper-cased before hashing
vehicle	Detects substitution of a different vehicle of similar description
odometer_km	Fixes the point in the vehicle's life at which the work was done
event_type	Separates a diagnosis from a completed repair
issuer_id	Records the workshop that sealed the record
timestamp	Orders events within one vehicle's history
report_hash	Any edit to the repair report breaks the digest
evidence_cid	Detects substitution of invoices, photographs or reports
model	Attributes the diagnosis to a model, adapter and training corpus
extra	Present even when empty, so later augmentation is visible

3.2 Merkle Tree and Root

Individual leaves are paired and hashed sequentially using a cryptographic hash function until only a single value, known as the Merkle root, remains [20], [31]. The root serves as a compact commitment to the entire batch, representing a fixed-size value that remains constant regardless of the number of repairs contained within. Two features make this construction particularly useful in this context. First, modifying any committed repair or evidence file alters the corresponding leaf digest, which cascades through every internal node on the path to the root, thereby changing the root; because the root was previously published, this alteration is detectable. Second, membership of a single repair can be verified without revealing the rest of the batch by providing only the sibling hashes along the path from that leaf to the root. A verifier independently recalculates the leaf from the provided record, applies the given path, and verifies that the resulting root matches the anchored root. This is the same construction that underlies transparency logs [32], [33].

3.3 Smart Contract and On-Chain Anchoring

The anchoring contract is intentionally small. It is written in Solidity for the 0.8.20 compiler series and stores one record type: a root, the block timestamp at which it was accepted, the batch identifier it belongs to, a metadata reference, and the address that submitted it. Two mappings index that record, one from root to anchor and one from batch identifier to root, so that a batch can be located either by the commitment it produced or by the name under which it was sealed.

One single external function does the anchoring, and it enforces four conditions before it writes anything. The root cannot be zero, the batch identifier cannot be empty, the root cannot already exist, and the batch identifier cannot already be bound to a different root. The third and fourth conditions are the ones that prevent reuse: a root cannot be anchored twice, and a batch identifier cannot be quietly re-pointed at a new root after the fact. Any of these attempts leads to a reversion, which is the behavior that the verification path relies on when it considers a recorded anchor to be final.

Each accepted anchor creates an event with the root, the batch identifier, the metadata reference, the submitting address and the timestamp, and the root and the submitter are indexed. Thus an observer who never queries the contract state can reconstruct the full anchoring history from logs alone, which matters for a verifier that holds only a record and a root and wishes to establish when that root first appeared.

One property should be declared in a straightforward manner, not found by a reader of the source. Anchoring is permissionless. The contract has an owner and an ownership transfer function, but the anchoring function is not restricted to the owner, so any address can anchor any root. The submitting address recorded with the root is therefore a claim about who sealed a batch, not an authorization to seal it. Establishing that a particular workshop made such a claim requires an identity layer the prototype does not implement, and the recorded address should be read accordingly. What the contract does guarantee is narrower but still useful: that a given root was present at a given block time, and has not been replaced since.

The deployment used for this work is a local development chain with chain id 31337, with the contract deployed at 0x5FbDB2315678afec. Four batches were anchored through it in the demonstration runs, each returning a transaction hash that the registry stores alongside the batch. This exercises the complete path including rejection of duplicate roots but confers no public persistence: a root anchored on a development chain is only as durable as that chain, which is why the same roots are submitted in parallel to a public timestamping service.

Table 2. Conditions enforced before a root is recorded

Condition	What it rejects
Root is non-zero	An empty or unset commitment
Batch identifier is non-empty	A batch recorded without a name
Root not already anchored	The same root being recorded a second time
Batch identifier unused	An identifier being re-pointed at a different root

3.4 OpenTimestamps

OpenTimestamps offers an independent timestamping path for the identical Merkle root [5], [35]. It is not a duplicate of the EVM anchor and should not be mistaken for one; instead, submissions are aggregated by public calendar servers into a commitment that is later included in the Bitcoin blockchain, while the receipt sent to the submitter remains pending until the aggregating transaction is confirmed, at which point it becomes a complete proof. The value of this

second path lies in the redundancy of trust. If the EVM deployment used for anchoring were abandoned or its history disputed, the OpenTimestamps receipt would still demonstrate that the root existed at the recorded time, and vice versa.

4. Evaluation

Model development employs parameter-efficient LoRA fine-tuning instead of full fine-tuning, ensuring that adaptation stays within a modest compute budget and the resulting adapters remain compact enough to be stored and versioned alongside the code [10], [44]. Two model paths are documented. The first approach adapts TinyLlama using an automotive and DTC-focused LoRA adapter [26]. The second approach adapts a 4-bit Mistral model using the MLX framework on Apple silicon [27], [45], following the quantized adaptation method [12]. Both paths are trained to output structured JSON diagnoses because the downstream commitment stage requires a parseable record, and the provenance of the model that generated a diagnosis is logged in the corresponding Proof-of-Repair leaf.

4.1 Diagnostic Results

The diagnostic layer was evaluated on a split disjoint by fault code, meaning that no code seen during training appears at evaluation. A fixed seed is used to split the corpus into 721 training, 86 validation and 163 test examples. The manifest describing the corpus is itself hashed and the resulting root is recorded in the provenance field of each sealed record.

On the primary hold-out of 57 real OBD cases, the 4-bit Mistral path adapted with MLX identified the correct fault code in 98.3% of cases and emitted a schema-complete record in 98.3%, failing outright once. The TinyLlama path with the automotive adapter achieved 93.0% on fault code while providing a valid record each time. It is the contrast that matters more than the gap. A pipeline that must hash its output treats an unparsable response as a repair that cannot be sealed, so perfect structural validity on a weaker model is not obviously worse than one failure on a stronger one.

TABLE 3. DIAGNOSTIC EVALUATION ACROSS ALL THREE EVALUATION SETS

Model	Evaluation set (n)	Fault code	Valid JSON	Severity
Mistral 4-bit	Real OBD hold-out (57)	98.3%	98.3%	59.7%
TinyLlama	Real OBD hold-out (57)	93.0%	100%	56.1%
Mistral 4-bit	Full DTC profile (163)	43.6%	96.9%	66.9%
TinyLlama	Full DTC profile (163)	32.5%	82.8%	60.1%
TinyLlama	Free-text complaints (106)	0.0%	73.6%	62.3%

Both figures decrease in situations for which the models have not been adapted. Mistral's fault-code accuracy drops to 43.6%, and TinyLlama's drops to 32.5% across the entire 163-case profile. The 106 free-text owner complaints in the national defects database are symptom descriptions with no quote of a diagnostic code, and fault-code accuracy on them is zero. The models learned to map codes to procedures, but not to infer codes from prose, and nothing in the training data asked them to do so. The figure of 98.3% would be misleading on its own.

Severity classification is weak, between 56% and 60% accuracy, and overlap with reference repair steps is low, with ROUGE-L scores of 0.13 to 0.24. These are both to be expected when there are several equally correct phrasings of a procedure, and neither is a safety claim. Within the hold-out, performance also splits by origin: Mistral labels every case from the instruction-tuned subset correctly, but is better calibrated on severity for the real workshop subset, 75.8% against 37.5%. This suggests the two corpora teach different things, and should not be pooled when reporting.

The attestation layer is reported separately because it has no accuracy. It either detects the change or it does not. Four batches were sealed, anchored on the development chain and timestamped, and each verification run against the stored records regenerated the committed digests. Every attempted alteration against them was detected.

A deployment condition is reported and not hidden. In a fresh deployment the sealing path fails before it reaches the model, because the trained adapter is not distributed with the implementation. This is a packaging gap rather than a defect in either layer, and it means the full workflow, including live inference, has not been shown end to end in that environment.

4.2 Case Study: Odometer Rollback

This is exactly the gap the prototype aims to fill, and the fraud serves as a useful test because the falsified quantity, or odometer reading, is one of the fields the system commits. Consequently, the scenario below was executed end-to-end against the actual implementation instead of being discussed in abstract terms. The repair event was finalized through the standard workshop workflow, batched, anchored, and verified; the committed odometer reading was then modified precisely as a rollback would, and the verification process was executed a second time to determine if the alteration was detected, what the buyer would observe, and what the results confirm or fail to confirm.

The case study utilizes a repair record for a 2016 Honda Civic (sample VIN: 1HGBH41JXMN109186) with an odometer reading of 98,000 km, which presented with fault code P0420, an illuminated check-engine light, a failed emissions test, and a sluggish rear oxygen sensor signal. The diagnostic context was submitted via the workshop bay form, after which the TinyLlama path, utilizing the automotive adapter, generated a structured repair report; subsequently, the report, its supporting evidence, and the model's provenance were sealed.

Sealing generated three values. The canonical repair report hashed to 44bd1f44aea82cdd, the stored evidence payload was written to the content-addressed store under the identifier d6faec167aef4751, and the Proof-of-Repair leaf that commits the vehicle identification number, vehicle description, odometer reading, report digest, evidence identifier, and model provenance was 1ecf00f00b0c9856. Although the leaf is small and holds no personal data, it commits every single one of those fields.

The leaf was subsequently batched with an additional event—a Toyota Camry record featuring the leaf ac512c77df8b6a8b—under the batch identifier ui-1788331032592. The pair was hashed to produce the Merkle root 46f5565e9a172ecb, which was anchored on the local development chain via transaction 0x22785f1ddd0d4b77 against the contract deployed at 0x5FbDB2315678afec, and simultaneously submitted to the public OpenTimestamps calendars under receipt 526ea1c4. At the time of writing, the receipt was still pending, which is the expected state for a live submission until the aggregating transaction is confirmed in a Bitcoin block, thereby illustrating the earlier point that the two anchoring paths are redundant rather than duplicated.

The stored record was then verified as an independent party would. The report digest, recomputed from the stored report, matched the committed value; the evidence identifier, derived from the stored bytes, matched the committed identifier; the leaf, reconstructed from the canonical payload, matched the committed leaf; and the membership proof, comprising the single sibling ac512c77df8b6a8b in this batch, successfully folded the leaf back to the anchored root. All four checks were successful.

One qualification must be stated clearly. Since the local development chain was not running at the time of writing, the on-chain check was not repeated; however, the anchor record in the registry preserves the transaction hash from the submission, and no stronger assertion is made beyond that. This stems from anchoring to a local chain instead of a public testnet—a limitation already noted in this paper—and is precisely the dependency that a public deployment would eliminate. The other four checks need only the record, the evidence file, and the published root, and can be executed by any party that possesses them.

The rollback was subsequently simulated against the sealed record. Changing the committed odometer reading from 98,000 to 76,000 kilometres, while keeping all other fields unchanged, updated the leaf digest from 1ecf00f00b0c9856 to 3ca71fb55475126d and recalculated the root from 46f5565e9a172ecb to 263fc73abe0b57aa. Since 63 out of the 64 hexadecimal characters in the root were different, the mismatch was immediately obvious without needing to read carefully. The membership proof generated for the original leaf also fails upon replay against the modified version, as the folding operation no longer yields the anchored value. A much smaller edit produces the same result: decreasing the reading by just one kilometre to 97,999 generates the completely unrelated leaf 93e8cccc8ccf6349 and root 9fc49570d5bd3e33. The design leaves no margin of error for an attacker to exploit.

The same record was run against two additional scenarios. The substitution of the stored evidence was detected without reference to the original file. Changing two bytes of the 1,966-byte invoice payload, so that the printed odometer now read 76,000, changed its content identifier from d6faec167aef4751 to 2c2f8a2364db9ef1, which corresponds to nothing that was anchored. The same was true for a real record reused on multiple vehicles: changing the Camry leaf ac512c77df8b6a8b to the Civic identification number, leaving all other fields untouched, moved the leaf digest to 63ab48af3bdad18c and the recomputed root to cc9b95ac3b703fb0, and the proof issued for the original leaf failed on replay. Because the vehicle identification number sits inside the hashed payload, a record cannot be moved between vehicles without the move becoming evident.

The principles established by the case should be stated narrowly. The buyer discovers that the record in question is the one that was sealed, and that it was sealed prior to the publication of the root. The buyer does not, however, discover that the diagnosis the record contains was correct: the report sealed here labelled fault code P0420 as a cooler-circuit fault and listed coolant-system steps, which deviates from the code's accepted meaning, and it was committed faithfully nonetheless. Nor does the buyer discover whether every repair performed on the vehicle was sealed at all, as the construction itself cannot testify to records that were never created. While integrity and completeness are distinct properties, only the former is asserted.

5. Limitations and Future Work

The prototype limits are provided as they stand. Anchoring runs on a local development chain, so roots are not publicly persisted and independent timestamps are pending until Bitcoin inclusion confirms them. The evidence is synthetic. No commercial workshop invoice has been ingested. No workshop partnership exists. The diagnostic context is entered rather than read from a scan tool, so the prototype is not recording anything directly from the vehicle. The evaluation is performed on 57 real OBD cases, sufficient to show feasibility and far too few to know how the models perform across different makes, production years and classes of fault. The models fail outright on free-text complaints. The registry is a single SQLite file next to a local evidence directory, which is fine for one writer and not concurrent multi-tenant operation. Workshop identity is unverified. Since anchoring is permissionless, the submitting address records a claim, not an authorization. A vehicle identification number is indirectly tied to an owner, so a deployed service would require a retention and disclosure policy even if committed leaves lack personal data.

Each of these leads to the future work. The next steps are to distribute the trained adapters with the implementation, so that a new deployment can perform inference without fetching them manually, and to publish the contract on a public test network, so that any party can verify a root without access to the development machine. Beyond those lie a larger evaluation corpus, grounding diagnoses in retrieved service documentation [17], pulling diagnostic context directly from OBD-II hardware, and adding workshop identity with role-based access control, before the system is given to real users instead of being run in a demonstration.

6. Conclusion

Repair history is worth little if it cannot be checked. This paper described a prototype to make it checkable by separating two things that are usually run together. A language model adapted by low-rank adaptation interprets a diagnostic context and generates a structured repair record. That record is tied to one vehicle with a cryptographic layer, and any change after that is obvious. Each sealed repair is a canonical leaf over the vehicle identification number, the odometer reading, the report digest, the evidence identifier, and the model provenance, and those leaves are collected into a Merkle tree whose root is then anchored on an EVM contract and timestamped independently.

The diagnostic layer was able to detect the fault code in 98.3% of real OBD cases, but that accuracy was completely lost on free-text complaints it had not been adapted for. The attestation layer successfully identified all attempts to tamper with sealed records, including a one-kilometre odometer change, a two-byte invoice edit, and the reuse of a record on different vehicles. The guarantee that accompanies it is deliberately limited: that the record is the one that was sealed, not that the diagnosis it contains is correct.

REFERENCES

Journals

[1] G. A. Akerlof, "The market for lemons: Quality uncertainty and the market mechanism," The Quarterly Journal of Economics, vol. 84, no. 3, pp. 488–500, 1970.

[2] B. Notheisen, J. B. Cholewa, and A. P. Shanmugam, "Trading real-world assets on blockchain: An application of trust-free transaction systems in the market for lemons," Business & Information Systems Engineering, vol. 59, no. 6, pp. 425–440, 2017.

[3] A. Dorri, M. Steger, S. S. Kanhere, and R. Jurdak, "BlockChain: A distributed solution to automotive security and privacy," IEEE Communications Magazine, vol. 55, no. 12, pp. 119–125, 2017.

[4] M. Cebe, E. Erdin, K. Akkaya, H. Aksu, and S. Uluagac, "Block4Forensic: An integrated lightweight blockchain framework for forensics applications of connected vehicles," IEEE Communications Magazine, vol. 56, no. 10, pp. 50–57, 2018.

[5] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," Journal of Cryptology, vol. 3, no. 2, pp. 99–111, 1991.

[6] Z. Zheng, S. Xie, H.-N. Dai, X. Chen, and H. Wang, "Blockchain challenges and opportunities: A survey," International Journal of Web and Grid Services, vol. 14, no. 4, pp. 352–375, 2018.

[7] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," Mechanical Systems and Signal Processing, vol. 104, pp. 799–834, 2018.

[8] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," Reliability Engineering & System Safety, vol. 215, art. 107864, 2021.

[9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

Conferences

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proc. International Conference on Learning Representations (ICLR), 2022.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in Proc. 36th International Conference on Machine Learning (ICML), 2019, pp. 2790–2799.

[12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NIPS), 2017, pp. 5998–6008.

[14] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38–45.

[15] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8026–8037.

[16] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[17] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[18] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. International Conference on Prognostics and Health Management (PHM), 2008, pp. 1–9.

[19] S. Matzka, "Explainable artificial intelligence for predictive maintenance applications," in Proc. 3rd International Conference on Artificial Intelligence for Industries (AI4I), 2020, pp. 69–74.

[20] R. C. Merkle, "A digital signature based on a conventional encryption function," in Advances in Cryptology — CRYPTO '87, Lecture Notes in Computer Science, vol. 293, Springer, 1988, pp. 369–378.

Books

[21] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction. Princeton, NJ: Princeton University Press, 2016.

[22] A. M. Antonopoulos and G. Wood, Mastering Ethereum: Building Smart Contracts and DApps. Sebastopol, CA: O'Reilly Media, 2018.

Standards, Specifications and Technical Reports

[23] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," white paper, 2008.

[24] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[25] J. Benet, "IPFS — Content addressed, versioned, P2P file system," arXiv preprint arXiv:1407.3561, 2014.

[26] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," arXiv preprint arXiv:2401.02385, 2024.

[27] A. Q. Jiang et al., "Mistral 7B," arXiv preprint arXiv:2310.06825, 2023.

[28] ISO 3779:2009, Road vehicles — Vehicle identification number (VIN) — Content and structure, International Organization for Standardization, Geneva, Switzerland.

[29] ISO 15031-5:2015, Road vehicles — Communication between vehicle and external equipment for emissions-related diagnostics — Part 5: Emissions-related diagnostic services, International Organization for Standardization, Geneva, Switzerland.

[30] SAE J1979, E/E Diagnostic Test Modes, SAE International, Warrendale, PA.

[31] Secure Hash Standard (SHS), FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg, MD, 2015.

[32] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, Internet Engineering Task Force, 2013.

[33] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency version 2.0," RFC 9162, Internet Engineering Task Force, 2021.

[34] IETF Supply Chain Integrity, Transparency and Trust (SCITT) Working Group, "An architecture for trustworthy and transparent digital supply chains," Internet-Draft, work in progress.

Web and Official Technical Sources

[35] P. Todd, "OpenTimestamps: Scalable, trustless, distributed timestamping with Bitcoin." [Online]. Available: https://opentimestamps.org

[36] National Highway Traffic Safety Administration, "Office of Defects Investigation (ODI) complaints database." [Online]. Available: https://www.nhtsa.gov

[37] NASA Prognostics Center of Excellence, "Turbofan engine degradation simulation data set (C-MAPSS)," NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA.

[38] S. Matzka, "AI4I 2020 predictive maintenance dataset," UCI Machine Learning Repository, 2020. [Online]. Available: https://archive.ics.uci.edu

[39] Ethereum Foundation, "Solidity documentation." [Online]. Available: https://docs.soliditylang.org

[40] Nomic Foundation, "Hardhat: Ethereum development environment." [Online]. Available: https://hardhat.org

[41] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[42] Encode, "Uvicorn: An ASGI web server implementation for Python." [Online]. Available: https://www.uvicorn.org

[43] SQLite, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html

[44] Hugging Face, "PEFT: Parameter-efficient fine-tuning." [Online]. Available: https://huggingface.co/docs/peft

[45] Apple, "MLX: An array framework for Apple silicon." [Online]. Available: https://github.com/ml-explore/mlx

[46] NBC News, "Odometer fraud rises as record number of Americans seek used cars," NBC Nightly News, 1 Apr. 2023. [Online]. Available: https://www.nbcnews.com

[47] CARFAX, "Odometer rollbacks see dramatic jump of 14% to 2.45 million vehicles," press release, 19 Dec. 2025. [Online]. Available: https://www.carfax.com

[48] National Highway Traffic Safety Administration, "Odometer fraud," U.S. Department of Transportation. [Online]. Available: https://www.nhtsa.gov/equipment/odometer-fraud